Daffodil International University

Lab Report Submission

Course Code: CSE 314

Course Name: Compiler Design Lab

Experiment No: 02

Experiment Name: Introduction to Flex and Bison — Lexical Analyzer, Arithmetic Expression Parser, Variable Assignment Parser, Boolean Expression Parser, If-Else Statement Parser, and List of Numbers Parser

Submitted To

Name: Rabeya Khatun

Designation: Lecturer

Department: CSE

Daffodil International University

Submitted By

Name: MD. Emon Hossain

ID: 232-15-818

Section: 66_L2

Semester: Spring 2026

Department: CSE

Daffodil International University

Task 1: Basic Arithmetic Expression Parser

1. Objective

Design a parser to evaluate arithmetic expressions with correct operator precedence and associativity using Lex and Bison.

2. Requirements

- Support operators: +, -, *, /
- Support parentheses for grouping
- Follow correct operator precedence rules
- Evaluate expressions using semantic actions

3. Theory

Bison is a parser generator that converts a Context-Free Grammar (CFG) into a C program that performs syntax analysis. It works together with Flex, where Flex handles tokenization and Bison handles parsing using an LALR(1) parser. Operator precedence is enforced through a three-level grammar: expr handles + and −, term handles * and /, and factor handles numbers and parenthesized sub-expressions.

4. Source Code

Lex File (calc.l) — Lexical Analyzer

```
%{
#include "calc.tab.h"
%}
%%
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
[ \t] ;
[+\-*/\(\)] return yytext[0];
\n return '\n';
. return yytext[0];
%%
int yywrap() { return 1; }
```

Bison File (calc.y) — Parser

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token NUMBER
%%
```

```
input:
    input expr '\n' { printf("= %d\n", $2); }
    | /* empty */
    ;
expr:
    expr '+' term { $$ = $1 + $3; }
    | expr '-' term { $$ = $1 - $3; }
    | term
    ;
term:
    term '*' factor { $$ = $1 * $3; }
    | term '/' factor { $$ = $1 / $3; }
    | factor
    ;
factor:
    NUMBER
    | '(' expr ')' { $$ = $2; }
    ;
%%
void yyerror(const char *s){ printf("Syntax Error: %s\n", s); }
int main() { printf("Enter expression:\n"); yyparse(); return 0; }
```

5. Compile & Run Commands

```
bison -d calc.y
flex calc.l
gcc lex.yy.c calc.tab.c -o calc
./calc
```

6. Grammar Rules Explanation

The grammar uses three non-terminals to enforce standard operator precedence:

- expr: handles addition (+) and subtraction (−) — lowest precedence
- term: handles multiplication (*) and division (/) — medium precedence
- factor: handles numbers and parenthesized expressions — highest precedence

This hierarchy ensures that * and / are evaluated before + and −, and parentheses override all precedence rules.

7. Test Cases & Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS    QUERY RESULTS

PS D:\Compiler-Design-CSE314> cd "Flex_&_Bison_Lab/Lab_Task_02/Task_01"
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_01> bison -d calc.y
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_01> flex calc.l
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_01> gcc lex.yy.c calc.tab.c -o calc
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_01> ./calc
Enter expression:
3+4*5
= 23
(2+3)*4
= 20
10 / 2 + 3
= 8
(8 - 2) * (3 + 1)
= 24
```

8. Result

The arithmetic expression parser was successfully implemented. The parser correctly evaluated all expressions while respecting standard operator precedence (* and / before + and −) and handled parentheses for grouping.

9. Conclusion

This task demonstrated how Flex and Bison cooperate to form the lexical analysis and syntax analysis phases of a compiler. The grammar-based approach of Bison ensures correct precedence and associativity in expression parsing.

Task 2: Variable Assignment Parser with Symbol Table

1. Objective

Implement variable assignment parsing and maintain a symbol table to store and retrieve variable values during expression evaluation.

2. Requirements

- Support single-letter variable names as identifiers
- Store and retrieve variable values using a symbol table
- Handle assignment statements of the form: ID = expr;
- Evaluate expressions involving previously assigned variables

3. Theory

A symbol table is a data structure used by compilers to store information about identifiers. In this experiment, a simple array sym[26] maps each lowercase letter (a–z) to its integer value. The parser recognizes assignment statements and stores the evaluated result in the symbol table for reuse in subsequent expressions.

4. Source Code

Lex File (assign.l) — Lexical Analyzer

```
%{
#include "assign.tab.h"
%}
%%
[a-z]   { yylval = yytext[0] - 'a'; return ID; }
[0-9]+  { yylval = atoi(yytext); return NUMBER; }
"="    return '=';
";"    return ';';
[+\-]  return yytext[0];
[ \t\r\n] ;
.       return yytext[0];
%%
int yywrap() { return 1; }
```

Bison File (assign.y) — Parser

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
int sym[26];
%}
%token ID NUMBER
%%
program: program stmt | stmt ;
stmt:
    ID '=' expr ';'
    { sym[$1] = $3; printf("%c = %d\n", $1 + 'a', $3); }
    ;
expr:
    expr '+' expr { $$ = $1 + $3; }
    | expr '-' expr { $$ = $1 - $3; }
    | NUMBER
    | ID { $$ = sym[$1]; }
    ;
%%
void yyerror(const char *s){ printf("Error: %s\n", s); }
int main() { printf("Enter assignments:\n"); yyparse(); return 0; }
```

5. Compile & Run Commands

```
bison -d assign.y
flex assign.l
gcc lex.yy.c assign.tab.c -o assign
./assign
```

6. Grammar Rules Explanation

- program: Recursively handles multiple statements.
- stmt: Parses ID = expr; and stores the evaluated value into sym[ID].
- expr: Supports addition, subtraction, numeric literals, and variable lookup via sym[ID].

7. Test Cases & Output

```
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_02> flex assign.l
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_02> bison -d assign.y
assign.y: conflicts: 4 shift/reduce
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_02> gcc lex.yy.c assign.tab.c -o assign
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_02> ./assign
Enter assignments:
x = 10;
x = 10
y = x - 4;
y = 6
a = 5;
a = 5
b = a + 3;
b = 8
a = a + b;
a = 13
```

8. Result

The variable assignment parser was successfully implemented. The symbol table correctly stored and retrieved variable values, and expressions involving previously assigned variables were evaluated as expected.

9. Conclusion

This task demonstrated how a symbol table can be implemented alongside a parser to support variable assignments. The Bison grammar rules directly manipulate the symbol table through semantic actions, illustrating how meaning (semantics) is attached to syntax during compilation.

Task 3: Boolean Expression Parser

1. Objective

Parse and evaluate boolean expressions containing relational operators (<, >, ==, !=) and logical operators (AND, OR, NOT), returning TRUE or FALSE.

2. Requirements

- Support relational operators: <, >, ==, !=
- Support logical operators: AND, OR, NOT
- Implement operator precedence for boolean operators
- Evaluate expressions returning 1 (TRUE) or 0 (FALSE)

3. Theory

Boolean expressions use logical operators AND (&&), OR (||), and NOT (!). Operator precedence for boolean logic is: NOT has the highest precedence, then AND, then OR. In Bison, this is declared explicitly using %right NOT, %left AND, %left OR directives. Relational operators produce integer results (0 or 1) which are consumed by the logical operators. Bison's %left and %right declarations resolve shift/reduce conflicts automatically.

4. Source Code

Lex File (bool.l) — Lexical Analyzer

```
%{
#include "bool.tab.h"
%}
%%
"AND"   return AND;
"OR"    return OR;
"NOT"   return NOT;
"TRUE"  { yylval = 1; return BVAL; }
"FALSE" { yylval = 0; return BVAL; }
[0-9]+  { yylval = atoi(yytext); return NUMBER; }
"=="   return EQ;
"!="   return NEQ;
"<"    return LT;
">"    return GT;
"("    return '(';
")"    return ')';
\n      return '\n';
[ \t]  ;
.       { return yytext[0]; }
%%
int yywrap(){ return 1; }
```

Bison File (bool.y) — Parser

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token AND OR NOT BVAL NUMBER EQ NEQ LT GT
%left OR
%left AND
%right NOT
%%
input:
    input expr '\n' { printf("Result: %s\n", $2 ? "TRUE" : "FALSE"); }
    | expr '\n'     { printf("Result: %s\n", $1 ? "TRUE" : "FALSE"); }
    ;
expr:
    expr AND expr    { $$ = $1 && $3; }
    | expr OR expr   { $$ = $1 || $3; }
    | NOT expr       { $$ = !$2; }
    | NUMBER LT NUMBER  { $$ = $1 < $3; }
    | NUMBER GT NUMBER  { $$ = $1 > $3; }
    | NUMBER EQ NUMBER  { $$ = $1 == $3; }
    | NUMBER NEQ NUMBER { $$ = $1 != $3; }
    | BVAL           { $$ = $1; }
    | '(' expr ')'   { $$ = $2; }
    ;
%%
void yyerror(const char *s){ printf("Syntax Error: %s\n", s); }
int main() { printf("Enter expression:\n"); yyparse(); return 0; }
```

5. Compile & Run Commands

```
bison -d bool.y
flex bool.l
gcc bool.tab.c lex.yy.c -o bool
./bool
```

6. Grammar Rules Explanation

- Relational operators (LT, GT, EQ, NEQ) compare two numbers and produce 0 or 1.
- AND, OR, NOT operate on boolean values with standard short-circuit semantics.
- Precedence: NOT > AND > OR ensures correct evaluation order without ambiguity.
- Parentheses group sub-expressions and override default precedence.

7. Test Cases & Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS    QUERY RESULTS

PS D:\Compiler-Design-CSE314> cd "Flex_&_Bison_Lab/Lab_Task_02/Task_03"
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_03> bison -d bool.y
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_03> flex bool.l
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_03> gcc lex.yy.c bool.tab.c -o bool
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_03> ./bool
Enter expression:
3 < 5 AND 4 > 2
Result: TRUE
NOT (3 > 5)
Result: TRUE
TRUE AND NOT FALSE
Result: TRUE
(TRUE OR FALSE) AND FALSE
Result: FALSE
```

8. Result

The boolean expression parser was successfully implemented. Expressions with relational and logical operators were correctly evaluated, and parenthesized grouping was handled as expected.

9. Conclusion

This task demonstrated the use of Bison precedence declarations (%left, %right) to resolve ambiguities in boolean expression grammars. The combined Flex-Bison system successfully performed tokenization and parsing of both relational and logical expressions.

Task 4: If-Else Statement Parser

1. Objective

Implement parsing of conditional if-else statements, handle nested conditions, and resolve the dangling-else ambiguity using Bison precedence directives.

2. Requirements

- Support IF-ELSE constructs
- Handle nested if-else cases
- Resolve the dangling-else ambiguity
- Execute the correct branch based on the condition value

3. Theory

The if-else statement is a fundamental control flow construct. When parsing nested if-else statements, the dangling-else problem arises: an else clause may match either the nearest or the outermost if. Bison resolves this using %nonassoc IFX and %nonassoc ELSE directives, giving ELSE higher precedence so it binds to the nearest if. The %union directive supports both integer (num) and character (id) token types in the same parser.

4. Source Code

Lex File (ifelse.l) — Lexical Analyzer

```
%{
#include "ifelse.tab.h"
#include <stdlib.h>
%}
%%
"if"    return IF;
"else"  return ELSE;
[a-z]   { yylval.id = yytext[0]; return ID; }
[0-9]+  { yylval.num = atoi(yytext); return NUMBER; }
"="    return '=';
";"    return ';';
"("    return '(';
")"    return ')';
[ \t\n] ;
.      return yytext[0];
%%
int yywrap(){ return 1; }
```

Bison File (ifelse.y) — Parser

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
int sym[26];
char last_id;
%}
%union { int num; char id; }
%token IF ELSE
%token <id> ID
%token <num> NUMBER
%type <num> expr stmt
%nonassoc IFX
%nonassoc ELSE
%%
program: program stmt | stmt ;
stmt:
    IF '(' expr ')' stmt %prec IFX
    { if($3){ printf("%c = %d\n", last_id, $5); } }
    | IF '(' expr ')' stmt ELSE stmt
    {
        if($3) printf("%c = %d\n", last_id, $5);
        else   printf("%c = %d\n", last_id, $7);
    }
    | ID '=' NUMBER ';'
    { last_id = $1; sym[$1-'a'] = $3; $$ = $3; }
    ;
expr: NUMBER { $$ = $1; } ;
%%
void yyerror(const char *s){ printf("Error: %s\n", s); }
int main(){ printf("Enter statements:\n"); yyparse(); return 0; }
```

5. Compile & Run Commands

```
bison -d ifelse.y
flex ifelse.l
gcc ifelse.tab.c lex.yy.c -o ifelse
./ifelse
```

6. Grammar Rules Explanation

- %nonassoc IFX and %nonassoc ELSE resolve the dangling-else ambiguity by making ELSE bind to the nearest if.
- %prec IFX lowers the precedence of standalone IF so that ELSE always attaches correctly.

- %union allows the parser to use both integer and character token value types simultaneously.
- The symbol table array sym[26] stores variable values indexed by letter position.

7. Test Cases & Output

```
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_4> gcc lex.yy.c ifelse.tab.c -o ifelse
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_4> ./ifelse
Enter statements:
if (1) a = 10; else a = 20;
a = 10
if (0) a = 10; else a = 20;
a = 20
if (1) b = 5; else b = 99;
b = 5
```

8. Result

The if-else statement parser was successfully implemented. The dangling-else ambiguity was correctly resolved using %nonassoc directives, and the conditional execution of assignment statements produced correct outputs for both branches.

9. Conclusion

This task demonstrated how Flex and Bison can be used to parse control flow structures. The use of %union, %nonassoc, and %prec directives in Bison is essential for handling ambiguous grammars and mixed token value types in real-world programming language parsers.

Task 5: List of Numbers Parser

1. Objective

Parse a comma-separated list of integers, count the number of elements, store them in an array, and print each parsed number and the total count.

2. Requirements

- Accept multiple numbers separated by commas
- Define a left-recursive grammar for list parsing
- Count the number of elements in the list
- Print each parsed number and the total count

3. Theory

A list grammar is a simple recursive Context-Free Grammar that recognizes sequences of items separated by delimiters. In Bison, left-recursive rules are preferred because they are more efficient for LALR(1) parsing and do not cause stack overflow for long lists, unlike right-recursive rules. The grammar recognizes comma-separated integers and processes each one through semantic actions.

4. Source Code

Lex File (list.l) — Lexical Analyzer

```
%{
#include "list.tab.h"
%}
%%
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
","   return ',';
[ \t\r\n] ;
.     return yytext[0];
%%
int yywrap() { return 1; }
```

Bison File (list.y) — Parser

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
int count = 0;
int arr[100];
%}
%token NUMBER
%%
list:
    list ',' NUMBER
    { arr[count++] = $3; printf("Parsed: %d\n", $3); }
    | NUMBER
    { arr[count++] = $1; printf("Parsed: %d\n", $1); }
    ;
%%
void yyerror(const char *s){ printf("Error: %s\n", s); }
int main(){
    yyparse();
    printf("Total elements: %d\n", count);
    return 0;
}
```

5. Compile & Run Commands

```
bison -d list.y
flex list.l
gcc list.tab.c lex.yy.c -o list
./list
```

6. Grammar Rules Explanation

- list ',' NUMBER: Left-recursive rule — appends a new number to the list, stores in arr[], and increments count.
- NUMBER: Base case — handles the first element of the list.
- Left recursion is preferred in LALR(1) parsers for efficiency and to avoid stack overflow on long lists.
- The count variable and arr[] array track the number and values of parsed elements.

7. Test Cases & Output

```
PS D:\Compiler-Design-CSE314> cd "Flex_&_Bison_Lab/Lab_Task_02/Task_5"
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_5> flex list.l
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_5> bison -d list.y
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_5> gcc lex.yy.c list.tab.c -o list
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_5> ./list
1,2,3,4,5
Parsed: 1
Parsed: 2
Parsed: 3
Parsed: 4
Parsed: 5
^Z
Total elements: 5
PS D:\Compiler-Design-CSE314\Flex_&_Bison_Lab\Lab_Task_02\Task_5>
```

8. Flex-Only vs Flex + Bison Comparison

Aspect	Flex Only	Flex + Bison
Scope	Lexical Analysis only	Lexical + Syntax Analysis
Grammar Support	Regular Grammars only	Context-Free Grammars
Expression Evaluation	Not possible	Yes — via semantic actions
Complexity Handling	Limited	Full language constructs
Use Case	Tokenizer / Scanner	Full compiler front-end

9. Result

The list of numbers parser was successfully implemented. The comma-separated input was correctly parsed, each number was printed and stored, and the total element count was reported accurately.

10. Conclusion

This task demonstrated the use of recursive Bison grammar rules for parsing structured data such as comma-separated lists. Combined with the Flex scanner, the Flex-Bison toolchain provides a complete and efficient solution for implementing the front end of a compiler or interpreter.